Embedded Systems Project 2
Improvements and Additional Features

EENG1030

Conor King – C22303056

Last Updated 05/05/2026

Contents

1. Abstract 4

This project presents a motion-based reaction game implemented on an STM32 microcontroller. The system uses an accelerometer to detect directional input and challenges the user to respond within a time constraint. Visual feedback is provided via an OLED display and RGB LED, while a high-score system tracks performance. Additional features include timeout logic, neutral position filtering, and low-power idle operation. 4

2. Introduction 4

Embedded systems are widely used in interactive applications requiring real-time input processing. This project builds on previous work by developing a motion-controlled reaction game using an STM32 platform. The system integrates multiple peripherals including an accelerometer, display, and LED indicators to create an interactive user experience. The objective was to design a responsive system with clear feedback, timing constraints, and efficient operation. 4

3. System Overview 4

4. Hardware Description (Fig. 1) 4

STM32 Nucleo (Controller) 4

Accelerometer (I2C) 4

OLED Display (SPI) 5

Push Button 5

RGB LED 5

Speaker / Audio Module (Experimental) 5

Breadboard and Wiring 6

5. Circuit Schematic (Fig. 2) 7

6. Troubleshooting and Problem Solving 7

7. Software Design 8

7.1 Timing Constants 8

7.2 Button Start Logic 8

7.3 Random Action Generation 9

7.4 Accelerometer Tilt Detection 9

7.5 Neutral Zone Filtering 9

7.6 Timeout Logic 10

7.7 High Score Logic 10

7.8 RGB LED State Feedback 10

7.9 Low-Power Idle Mode 10

7.10 Audio Sample Array 11

7.11 Audio Playback Attempt 11

7.12 I2S Bit Transmission 11

7.13 Audio Limitation 11

Figure 1 Image of Real-Life Breadboard 6
Figure 2 Schematic Design 7

1. Abstract

This project presents a motion-based reaction game implemented on an STM32 microcontroller. The system uses an accelerometer to detect directional input and challenges the user to respond within a time constraint. Visual feedback is provided via an OLED display and RGB LED, while a high-score system tracks performance. Additional features include timeout logic, neutral position filtering, and low-power idle operation.

2. Introduction

Embedded systems are widely used in interactive applications requiring real-time input processing. This project builds on previous work by developing a motion-controlled reaction game using an STM32 platform. The system integrates multiple peripherals including an accelerometer, display, and LED indicators to create an interactive user experience. The objective was to design a responsive system with clear feedback, timing constraints, and efficient operation.

3. System Overview

- User presses button → game starts
- Display shows direction
- User tilts device
- System checks correctness
- Score updates
- Timeout or wrong input ends game
- High score stored

4. Hardware Description (Fig. 1)

STM32 Nucleo (Controller)

- Acts as the central control unit of the system
- Executes all game logic, timing functions, and state transitions
- Interfaces with all peripherals (I2C, SPI, GPIO)
- Processes accelerometer data and determines user actions

The STM32 is responsible for coordinating all inputs and outputs in real time, ensuring that the system responds immediately to user interaction.

Accelerometer (I2C)

- Measures acceleration in X and Y axes
- Used to detect directional tilt (left, right, forward, back)
- Provides the primary user input mechanism
- Communicates with STM32 via I2C protocol

The accelerometer replaces traditional buttons by allowing motion-based control, forming the core interaction method of the game.

OLED Display (SPI)

- Displays game instructions (direction prompts)
- Shows current score and high score
- Provides real-time visual feedback to the user
- Communicates via SPI for fast data transfer

The display serves as the main user interface, allowing the player to understand system prompts and monitor performance.

Push Button

- Used to start the game
- Wakes the system from idle (low-power mode)
- Configured with internal pull-up resistor
- Provides a simple digital input to the STM32

The push button acts as the system trigger, ensuring controlled startup and preventing accidental activation.

RGB LED

- Provides quick visual indication of system state
- Colour-coded states:
    - Red → Idle / waiting
    - Blue → Game active
    - Green → End of round
- Controlled via GPIO outputs

The RGB LED provides immediate feedback without needing to look at the display, improving usability and responsiveness.

Speaker / Audio Module (Experimental)

- Intended to provide audio feedback for events such as high score
- Driven using digital signals from the STM32

- Attempted implementation using I2S (MAX98357A amplifier)
- Used sound data array for playback

The audio system was designed to enhance user feedback; however, due to strict timing requirements for I2S communication, reliable playback was not achieved. The module remains part of the design but is not critical to core functionality.

Breadboard and Wiring

- Used to prototype and interconnect all components
- Provides flexible layout for rapid testing and modification
- Distributes power (3.3 V and GND) across all modules

The breadboard enables quick hardware iteration and testing, which was essential during debugging and system integration.

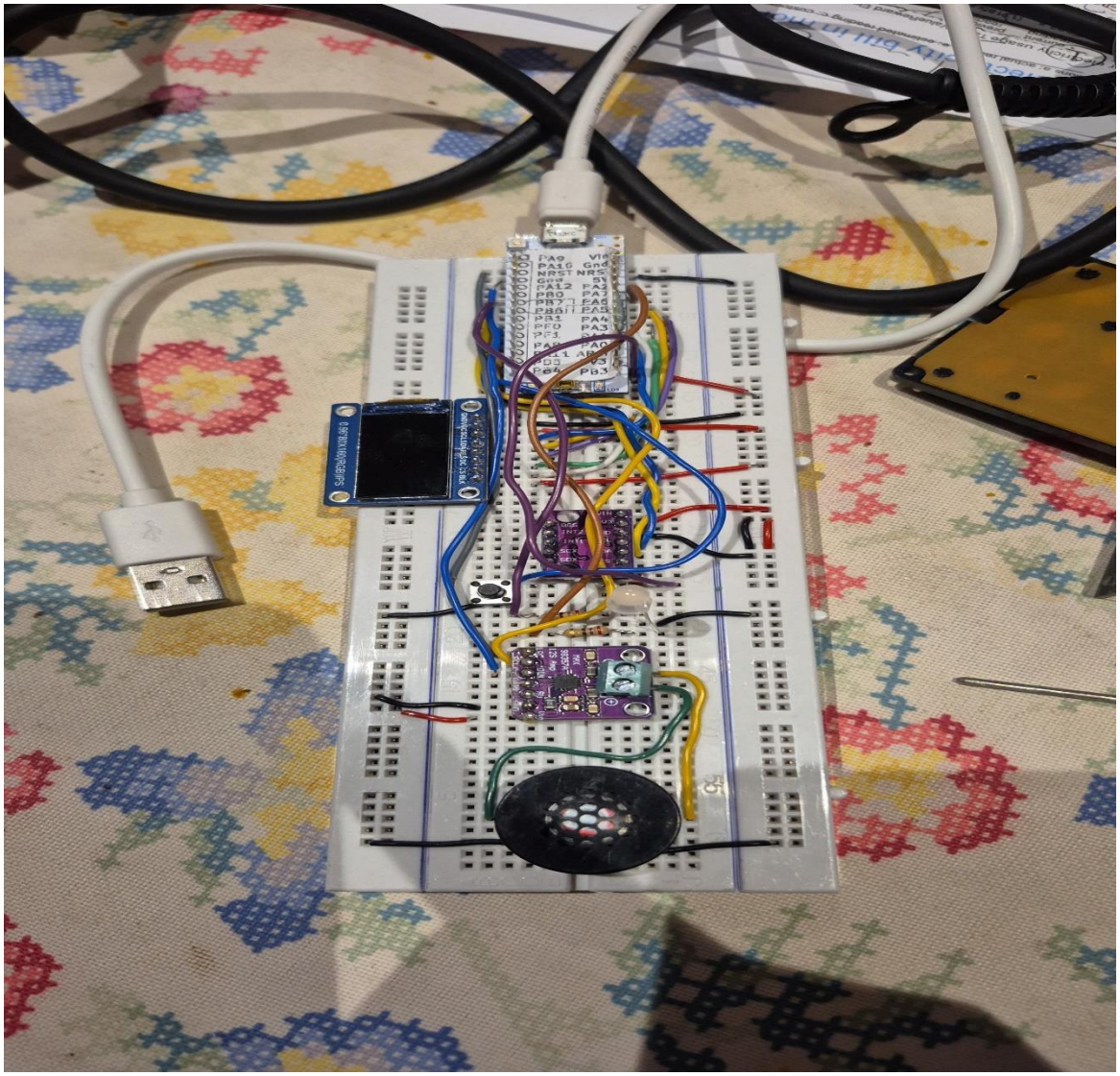

Figure 1 Image of Real-Life Breadboard

5. Circuit Schematic (Fig. 2)

The circuit schematic (Fig. 2) shows the interconnection between the STM32 microcontroller and all peripherals. Communication protocols include I2C for the accelerometer and SPI for the display. GPIO pins are used for button input, LED control, and audio output. Proper power and ground distribution are maintained across the system.

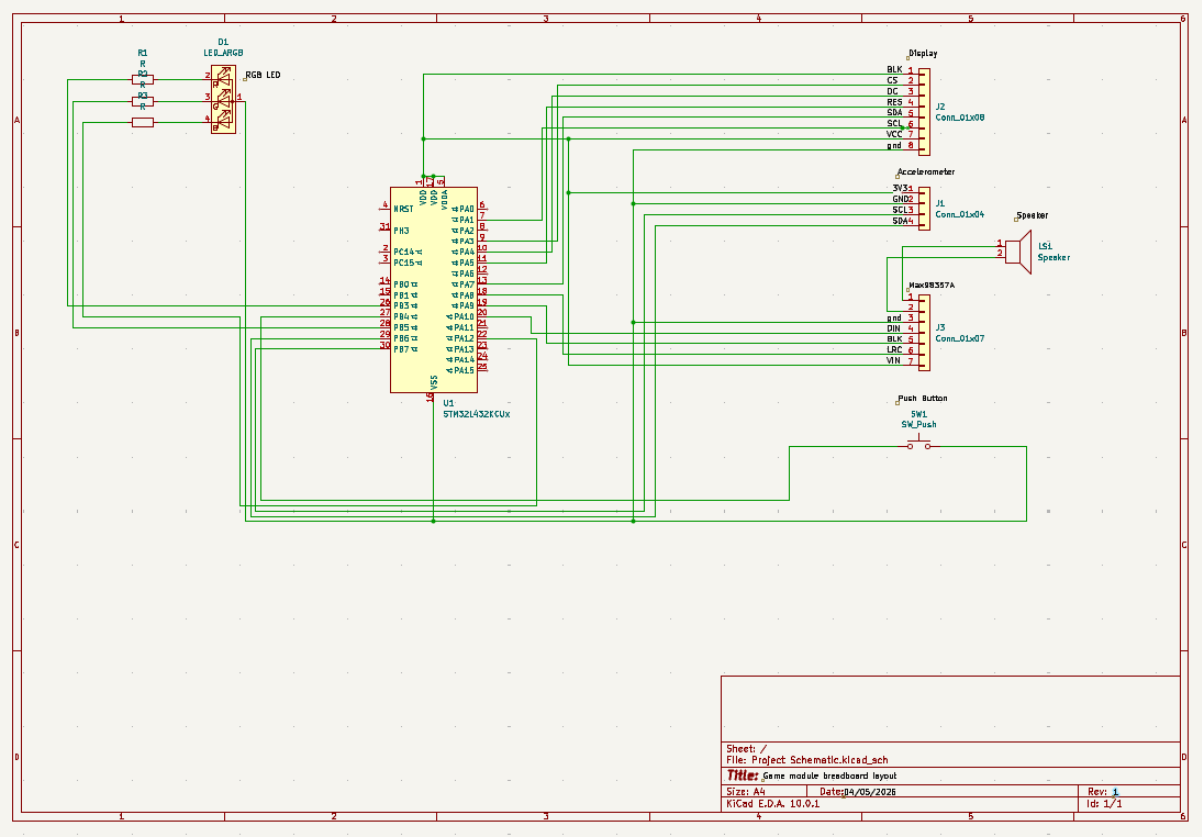

Figure 2 Schematic Design

6. Troubleshooting and Problem Solving

The system was developed using an incremental testing approach, where each component was verified individually before full integration. Basic electrical checks were first carried out using a multimeter to confirm correct voltage levels, common grounding, and continuity across the breadboard. Individual modules such as the push button, RGB LED, display, and accelerometer were then tested using small standalone code segments to ensure correct operation. This approach allowed faults to be isolated quickly and ensured that any issues encountered during full system integration were more likely related to software logic or communication timing rather than wiring errors.

During integration, timing-related behaviour and input reliability were refined through testing. The accelerometer thresholds were adjusted based on observed readings to ensure that only deliberate movements were detected, while a neutral zone was implemented to prevent repeated triggering. Buttons debounce delays were also introduced to eliminate false inputs. Visual feedback through the RGB LED was verified to correctly represent system states, and

timeout logic was tested to ensure the game ended correctly when no input was detected within the specified time window.

A significant issue was encountered with the speaker and audio output. The speaker was initially tested directly from a GPIO output, where no meaningful audio signal was produced. A sound array generated externally was then used to drive the speaker, but only a very weak or brief signal was observed. When the MAX98357A amplifier module was introduced, a small click or buzz was detected, indicating that some signal was present, but no clear audio playback was achieved. This suggests that while the data was being transmitted, the timing requirements for proper I2S communication were not being met. As a result, audio functionality was considered experimental and was not relied upon for the final system, although the testing process confirmed partial signal activity and ruled out complete hardware failure. Time was no doubt a factor here too and some attempts were used just with small segments of code to get the operation working but again it seemingly refused to work.

7. Software Design

The software was structured so that each major function of the system is handled separately. The main program controls the game flow, while separate functions are used for accelerometer reading, tilt detection, display updates, RGB LED feedback, low-power idle mode, and experimental audio playback.

7.1 Timing Constants

```
#define ROUND_TIME_MS     4000
#define IDLE_SLEEP_MS     10000
#define DEBOUNCE_MS       250
#define RESULT_DISPLAY_MS  2000
```

These constants define the main timing behaviour of the system. The player has four seconds to respond to each displayed instruction. A debounce delay is used to prevent one button press from being detected multiple times, and the result screen remains visible for two seconds before returning to idle.

7.2 Button Start Logic

```
button = (BUTTON_PORT->IDR & (1 << BUTTON_PIN)) ? 1 : 0;

if ((last_button == 1) && (button == 0) && !running)
{
   running = 1;
   score = 0;
   ready = 1;

   action = next_action();
   round_start_time = milliseconds;
}
```

The push button uses an internal pull-up resistor, meaning the input is normally high and becomes low when pressed. The code detects this falling edge and starts a new game only when the system is not already running.

7.3 Random Action Generation

```
static uint8_t next_action(void)
{
  static uint8_t count = 0;
  count++;

  return (uint8_t)((milliseconds + count * 7) % 4);
}
```

This function generates the next movement instruction. The system uses the current millisecond timer and an increasing counter to vary the output. The result is limited to four possible values: left, right, forward, and back.

7.4 Accelerometer Tilt Detection

```
static uint8_t detect_tilt(int32_t x, int32_t y)
{
  if (y < 200 && y > -200)
  {
    if (x > X_THRESHOLD) return ACT_RIGHT;
    if (x < -X_THRESHOLD) return ACT_LEFT;
  }

  if (x < 300 && x > -300)
  {
    if (y > Y_FORWARD_THRESHOLD) return ACT_FORWARD;
    if (y < Y_BACK_THRESHOLD) return ACT_BACK;
  }

  return ACT_NONE;
}
```

The accelerometer values are compared against threshold values to decide which direction the user has tilted the device. X-axis values are used for left and right movement, while Y-axis values are used for forward and back movement. If no threshold is exceeded, no action is detected.

7.5 Neutral Zone Filtering

```
if ((x < X_NEUTRAL_HIGH && x > X_NEUTRAL_LOW) &&
  (y < Y_NEUTRAL_HIGH && y > Y_NEUTRAL_LOW))
{
  ready = 1;
}
```

A neutral zone is used so that one tilt is not counted repeatedly. After a movement is detected, the user must return the board close to its neutral position before another input can be accepted.

7.6 Timeout Logic

```
if ((milliseconds - round_start_time) > ROUND_TIME_MS)
{
  running = 0;
  rgb_green();
  draw_fail(score, high);
}
```

The timeout logic ends the game if the user does not respond within the allowed time. This creates the reaction-game behaviour and prevents the system from waiting indefinitely for input.

7.7 High Score Logic

```
if (score > high)
{
  high = score;
  rgb_green();
  draw_high(score);
  play_sound_file();
}
```

The high score is stored during runtime. If the player beats the previous high score, the display updates to show the new score and the stored audio sample is attempted through the audio output system.

7.8 RGB LED State Feedback

```
rgb_red();   // idle / waiting
rgb_blue();  // game active
rgb_green(); // result screen
```

The RGB LED acts as a quick system status indicator. Red shows the system is idle, blue shows the game is active, and green shows the end/result state. This gives feedback even when the user is not focused on the display.

7.9 Low-Power Idle Mode

```
static void enter_idle_sleep(void)
{
  rgb_off();
  __WFI();
}
```

The system enters a light sleep state after a period of inactivity. The __WFI() instruction places the CPU into a wait-for-interrupt mode, reducing unnecessary activity while preserving variables such as the high score.

7.10 Audio Sample Array

```
#include "sound_array.h"

const int16_t sound_data[] = {
  3, 2, -1, -5, 5, -1,
  ...
};

const unsigned int sound_length =
  sizeof(sound_data) / sizeof(sound_data[0]);
```

The audio sample is stored in a separate header file as signed 16-bit values. These values were generated from an external audio file and converted into a C array. This allows the sound data to be stored directly in the microcontroller flash memory.

7.11 Audio Playback Attempt

```
static void play_sound_file(void)
{
  for (uint32_t i = 0; i < sound_length; i++)
  {
    int16_t s = sound_data[i] * 3;
    i2s_send_stereo(s, s);
  }
}
```

This function loops through the stored audio sample and sends each value to the audio output function. The sample is multiplied to increase amplitude before being sent to both left and right audio channels.

7.12 I2S Bit Transmission

```
static void i2s_send_stereo(int16_t left, int16_t right)
{
  I2S_LRC_PORT->ODR &= ~(1 << I2S_LRC_PIN);
  i2s_send_word(left);

  I2S_LRC_PORT->ODR |= (1 << I2S_LRC_PIN);
  i2s_send_word(right);
}
```

The MAX98357A audio module uses I2S communication, requiring separate signals for bit clock, left/right clock, and data. This function attempts to send left and right channel audio samples by manually controlling the GPIO pins.

7.13 Audio Limitation

Again, although the audio sample was successfully stored and transmitted in software, reliable playback was not achieved. The MAX98357A requires accurately timed I2S signals, and software-based bit-banging was not accurate enough to produce clear sound. It is still unclear what the issue was.